

SP3_HWv2.1 改用官方 MicroPython 韌體評估清單

對象 branch：SP3_HWv2.1 (智付小卡 v2.1 · RGB LED 版) | 產出時間：2026-09-07 15:55
現況韌體：MicroPython v1.18-9-gd8e35d0e0-dirty (2022-02-19) MPYBLOCKLY ESP32 (廠商客製版，該編譯公司已停止運作)
目標韌體：MicroPython v1.29.0 (2026-08-24) 官方通用 .bin (最新版；SP3 第一版即以此為底層)

決策背景：原廠商客製韌體 (v1.18.9 · 2022) 之編譯公司已停止運作，等同棄養的 4 年前版本，無法再取得底層更新。SP3 尚未出貨，屬全新產品線、零遷移成本，是換底層韌體最好的時機。因此決定 SP3 第一版即以官方 MicroPython v1.29.0 為底層 (詳見 G 節)。

查證聲明：相關事實已於 2026-08 上網查證官方 release notes 與文件 (來源見末尾)，並在備品板上實機驗證 (v1.29.0 與 1.18.9 各測)。標示 **已實測** 者為實機確認結果。

範圍：本評估假設 SP3 已移除 LCD、改用 rgb_led_manager.py (單顆 WS2812)，僅比較新舊 MicroPython 差異，不含任何 LCD 相關評估。

A. 凍結模組相容性 **已實測**

換官方通用 .bin 前，原本最擔心的是：現況廠商客製韌體把許多模組凍結(frozen)進韌體，換官方版後若這些模組不在檔案系統裡就會全部消失。經在備品板上以 help('modules') 實機查證，本專案用到的模組在官方 ESP32 v1.29.0 build 中也都已 frozen 內建，此風險實際上已解除。

模組	用途 / 位置	官方 ESP32 v1.29 .bin 是否內含	風險	對策
umqtt.simple	MQTT 通訊 (analogCoinPay_Main.py)	是 (官方已 frozen 內建)	低	無需處理
urequests	OTA 下載 (senko.py)	是 (官方已 frozen 內建)	低	無需處理
neopixel	RGB 狀態燈 (rgb_led_manager.py)	是 (官方內建)	低	無需處理
espnow	(未來 ESP-NOW 前瞻，見 G 節)	是 (官方內建 · v2.0)	低	無需處理
ntptime	NTP 校時	是，且另自帶本地 sourceFiles/ntptime.py	低	本地檔優先， 無需處理

好消息 (已實測修正先前評估)：先前評估曾誤判 umqtt.simple / urequests 需另補；經 help('modules') 實測，官方 ESP32 v1.29.0 build 已內建這兩者 (連 neopixel、espnow、ntptime 亦然)。加上 SP3 改用 RGB LED、顯示端只依賴官方內建的 neopixel，換官方韌體幾乎沒有要補的模組。

B. API / 命名相容性

B-1. u-前綴模組 (utime / uos / ujson / ure / uhashlib) **已實測**

低風險 (v1.29 實機已測)。自 v1.21.0 起官方將內建模組去除 u-前綴 (utime→time 等)，但保留 u-名稱作為相容別名，到 v1.29.0 都未移除。實機在 v1.29.0 上跑完整開機流程，各檔 import utime / uos / ujson / ure / uhashlib 均正常，無 ImportError。

- 現況可直接沿用 u-前綴寫法，v1.29 無須為此改動。
- 建議 (非緊急、與韌體升級無關)：日後逐步把 import 改為非前綴名 (time / os / json / re / hashlib)，與程式內既有的 binascii / socket / struct 一致，為官方未來真正移除 u-模組預作準備。

B-2. WiFi 主機名 API (dhcp_hostname) 已實測

低風險 (已實作並雙韌體實測通過) 。 wifimgr.py 原用 `self.wifi.config(dhcp_hostname=...)` 設 `SmartPay_xxxxxx` , 此法在官方新韌體已 deprecated 、設不上 (DHCP 主機名會變預設 `mpy-esp32`) 。已改為依韌體分流 , 兩者皆在 `connect()` 內、`wifi.connect()` 前一刻設定 :

韌體	偵測方式	設定方式 (實測有效)
新韌體 v1.29 (有 <code>network.hostname</code>)	<code>hasattr(network, 'hostname')</code>	<code>active(False) → sleep(1) → network.hostname(name) → active(True)</code> (name 須在 active 之「前」設)
舊韌體 1.18.9 (無 <code>network.hostname</code>)		<code>config(dhcp_hostname=name)</code> (須在 active 之「後」設)

- 實測結果 : 新舊韌體各測 , 手機 AP 端皆正確顯示 `SmartPay_xxxx` (MAC 末三碼一致) 。
- 此改動向後相容 , 可回饋 SP2 (LCD 版) 讓兩邊 wifimgr 一致 。

B-3. v1.29.0 明確 breaking changes 對本專案的影響

下列為 v1.29.0 引入的變更 (本文目標即 v1.29.0 , 故皆適用) 。經檢查 , 對本專案多半無影響 :

v1.29.0 變更	對 SP3 影響	說明
<code>str.encode()/bytes.decode()</code> 只接受 utf-8 / ascii	否	已檢查 : 程式一律用 'utf-8' 或預設值 , 無 latin-1 / 大寫寫法
SDCard readblocks/writeblocks 回傳值反轉	否	本專案未使用 SD 卡
nrf / STM32 Cortex-M33 相關	否	非 ESP32 , 與本專案無關
ESP32 IDF 更新、ESP-NOW、mDNS	否 (已實測基本行為)	底層更新 ; 實機開機 → WiFi → NTP → MQTT → OTA 檢查流程皆正常

C. RGB LED / 中斷 / 計時器

- neopixel 為官方內建 , machine.bitstream 驅動 ; 單顆 WS2812 一次 `write()` 約關中斷 30μs , 對 5ms 以上的投幣/電眼脈波影響可忽略 (正常脈波不遺失 , 僅 <30μs 雜訊可能被併掉 , 本就會被寬度判斷濾除) 。此點待娃娃機實機用脈波寬度 log 最終確認 (投幣 Low Pulse 設定值 ±10ms 內視為通過) 。
- LED tick 採被動式、不用 timer 也不用 thread : SP3 的 `rgb_led_manager` 不自帶 timer/thread , 改由主程式在既有節奏 (three-task 及各阻塞段) 呼叫 `tick()` 推進動畫 。 `tick()` 很短 (算一幀 + 最多一次 30μs write) , 完全避開「timer callback 與 GPIO soft-irq 共用排程器、導致中斷延後」的老問題 。
- ESP32 Timer(-1) 歷史 (供參考) : ESP32 早期 `Timer(-1)` 「能動」其實是無範圍檢查下巧合選到硬體 timer ; v1.26 起加檢查會拒絕 -1 ; v1.29 才真正實作虛擬(軟體) timer (`Timer(-1)` 可用) 。本專案 LED 採被動 tick , 兩者都不依賴 。
- 提醒 : `tick()/neopixel.write()` 不可在 ISR 內呼叫 ; LED 更新只放在正常節奏的主流程 。

D. 逐檔改動 Checklist (走官方韌體時)

檔案	要做的事	類型
(檔案系統)	無需補模組 : <code>umqtt</code> / <code>urequests</code> / <code>neopixel</code> 官方 ESP32 v1.29 build 已內建 (實測)	OK

sourceFiles/analogCoinPay_Main.py	umqtt.simple 實機可載；u-前綴沿用即可，日後再視情況遷移	OK
sourceFiles/main.py	u-前綴沿用即可；已改 RGB LED 初始化與 SW1 停止（見下）	OK
sourceFiles/wifimgr.py	主機名已改 hasattr 分流（新 network.hostname() / 舊 dhcp_hostname ），連線前設定	已實測
sourceFiles/senko.py	urequests 實機可載；uhashlib/utime/uos 沿用即可	OK
sourceFiles/rgb_led_manager.py	無需改（ neopixel 官方內建 ）	OK

實機驗收必測項：MQTT 連得上 **已測** | OTA(senko) 能更新 | WiFi 主機名有設上 **已測** | 投幣/電眼中斷不掉脈波（待娃娃機） | 各檔 import 無 ImportError **已測** | RGB 燈號十態正常（待目視）。

順帶已完成的兩項韌體相容改動：

- SW1 開機停止：main.py 由 sys.exit() 改為 raise KeyboardInterrupt。原因：v1.29 的 sys.exit()/SystemExit 會被當 forced-exit 觸發 soft reset → 停不進 REPL；改一般例外後，v1.29 與 1.18.9 皆實測乾淨掉到 REPL，且兩韌體通用、不需分支。
- u-模組 / 主機名：如 B-1、B-2，皆已雙韌體實測。

E. 補充問答 1：mip 是什麼？是做成 mpy 嗎？

不是。mip = 「mip installs packages」，是 MicroPython 在裝置上的套件安裝器，相當於 PC 上的 pip。

- 行為：從網路（micropython-lib 套件庫或指定 URL）下載模組，複製到裝置的 /lib 目錄。需要裝置能連網。
- 它下載回來的內容可能是 .mpy 也可能是 .py（依套件而定），但 mip 本身不負責編譯。
- 把 .py 編成 .mpy 的工具是另一個東西：mpy-cross（在 PC 上跑的交叉編譯器）。

對照：mip = 安裝器（抓檔案放進裝置）；mpy-cross = 編譯器（.py → .mpy）。兩者不同層次。本專案模組官方已內建，其實用不到 mip 去補。

F. 補充問答 2：現有 .py vs 用 v1.29 編成 .mpy，哪個好？

前提：假設你用對應 v1.29 的 mpy-cross 把源碼編成 .mpy，跑在 v1.29 韌體上。

面向	.py（裝置上即時編譯）	.mpy（v1.29 預編位元碼）
相容性	最寬鬆：只要語法/API 沒變，換韌體重新 import 即可用；不綁 bytecode 版本	綁 bytecode ABI：firmware 與 mpy-cross 版本必須一致；換韌體大版本要重編，否則 incompatible .mpy file
執行速度	import 後為同一套 bytecode，執行速度相同	執行速度相同；只有 import 當下較快（省去在裝置上編譯）
記憶體用量	import 時要在裝置上跑編譯器，峰值 RAM 高；大檔可能 MemoryError	較省 RAM：不需在裝置編譯，載入更精簡，大檔更穩
維護 / 開發	可直接編輯、直接 OTA，開發方便	改一次要重編一次；OTA 也要送 .mpy

對本專案的實務結論：

- 第三方 lib (umqtt 、 urequests) : 官方已內建、無需自帶；只有在你要覆寫成自訂版本時才需考慮，屆時 .mpy 省 RAM 較合適。
- 你自己常改的主程式：開發期用 .py 方便；但 analogCoinPay_Main.py 曾因「檔案太大、編譯吃 RAM 而跑不動」（README 有記載為此刪註解瘦身），這正是 .mpy 能解的問題——與 to-be-do 清單第 3.a 「讓 AI 改成可以跑 mpy」一致。
- 鐵則：mpy-cross 的版本必須與目標韌體版本一致，否則 .mpy 無法載入。

G. 決策與總建議

已決定：SP3 第一版以官方 MicroPython v1.29.0 為底層。
主因：面向未來以 ESP-NOW v2.0 預留機台間網路串接能力（詳見下方）。

為什麼現在換（而非沿用舊韌體）

- 原廠商已停止運作：v1.18.9 是棄養的 4 年前客製版，拿不到任何底層更新，繼續留在上面只會累積技術債、出問題也無人能修。
- SP3 尚未出貨、零遷移成本：換底層韌體最痛的是既有機台升級；SP3 是全新產品線沒有這包袱，第一版就上新韌體是成本最低的一刀。
- 面向未來：官方版才對得上網路上 / AI 產生的程式碼，利於日後新增 QR scanner、新 MQTT 指令；記憶體 headroom 與穩定性也優於 2022 舊版。

選 v1.29.0 的理由與代價 (vs v1.28.0 / v1.27.0)

版本	發布	取捨
v1.29.0 (選定)	2026-08-24	優點：最新修正、最新 IDF/ESP32 支援、ESP-NOW v2.0、真正的虛擬 timer Timer(-1)（本專案 LED 採被動 tick、未必用到）。代價：發布時點較新，早期回歸風險略高，須以徹底桌測補足（本專案已完成雙韌體桌測）。
v1.28.0	2026-04-06	成熟穩定 + 功能現代的平衡點，社群實測回報較多。缺虛擬 timer、ESP-NOW 僅 v1（250B）。
v1.27.0	2025-12-09	更保守，但相對 1.28 無明顯優勢，只是更舊。

版本取捨提醒：若最看重「出貨穩定度」，v1.28.0 是較保守選擇。本專案 LED 不依賴虛擬 timer，故該加分並非關鍵；選 1.29 的主要理由是下述的 ESP-NOW 前瞻——代價是版本較新，已用雙韌體桌測補足。

選 v1.29.0 的關鍵理由：ESP-NOW 前瞻

SP3 預留未來以 ESP-NOW 增強機台網路串接的可能性：當店面（尤其戶外、機台間距大、或本身網路差的娃娃機 / 遊戲機場）WiFi 訊號不佳時，可讓機台之間直接互傳 / 中繼，補強單機連網。v1.29.0 的 ESP-NOW 已是 v2.0（單封包上限 1470B、且修掉 peer 傳入 falsey 值的 crash），比 v1.28 的 ESP-NOW v1（250B）更適合當底層；先上 1.29 可避免日後為此重燒韌體。

未來導入 ESP-NOW 的三個共存前提（與版本無關，屆時務必遵守）：

- 頻道被 AP 綁死：同時連 AP(STA) 跑 MQTT 時，ESP-NOW 只能用 AP 所在頻道；多機互傳需在同一頻道，AP 自動換頻道會更複雜。
- 關閉 WiFi power-save，否則 ESP-NOW 接收會掉包。

- `recv callback` 保持輕量 (只收 `buffer` / 設 `flag`) , 重活丟別處 , 避免延後投幣 / 電眼中斷 (與 `timer`→被動 `tick` 同一紀律) 。

執行步驟

1. 先在備品板上驗 , 通過後才納入出廠燒錄流程。(韌體相容面已完成雙韌體桌測 ; 剩娃娃機實機的中斷 / 燈號驗收。)
2. A 節凍結模組經實測官方已內建、無需補 ; B 的 `network.hostname()` 分流與 u-模組相容性亦已實測。
3. OTA 不受影響 : 韌體是出廠燒一次的事 , `senko` OTA 只更新 `repo` 的 `.py` (`urequests` 官方已內建 , 無需打包) 。SP3 出廠統一燒 `v1.29.0` → 全機台一致、無新舊混雜。
4. 誠實提醒 : 跨版本主要改善的是穩定性與記憶體 `headroom` ; 「IO 中斷間隔算不準」多屬 ISR 設計問題 (現有 `ticks_diff` 溢位處理已正確) , 換韌體不會自動變準。

來源 (2026-08 查證)

- MicroPython v1.29.0 release : <https://github.com/micropython/micropython/releases/tag/v1.29.0>
- u-模組去前綴說明 (FYI #12078 / v1.21.0 notes) : <https://github.com/orgs/micropython/discussions/12078>
- `network` 模組文件 (`hostname`) : <https://docs.micropython.org/en/latest/library/network.html>
- `dhcp_hostname` deprecated 討論 : <https://github.com/orgs/micropython/discussions/11652>
- ESP32 下載頁 : <https://micropython.org/download/esp32/>
- 凍結模組清單 : 備品板 `help('modules')` 實機查證 (2026-08)